

UNIT - II

Introduction to MongoDB

End Semester Examination Notes

Covers: Why MongoDB | What is MongoDB | Documents & Collections | Data Model Design

1. The Need of MongoDB, Its Use & Its Benefits

1.1 What is a Database?

In a data-driven world, we need organized and easily accessible information. A database is any structured information or data specially organized and stored in a computer for fast retrieval and search.

There are two primary types of databases:

- SQL (Relational) — stores data in tables with rows and columns. Examples: Microsoft SQL Server, Oracle, Sybase.
- NoSQL (Non-Relational) — stores schema-less, unstructured data in multiple collections and nodes. Does not need fixed table schemas.

1.2 MongoDB Evolution

Understanding how MongoDB came to be helps explain why it exists:

1. SQL existed even before the World Wide Web. However, as website functionality grew, developers wanted to generate web pages using content that could change over time without re-deploying the code.
2. NoSQL has relaxed ACID (Atomicity, Consistency, Isolation, Durability) properties and guarantees better performance, scalability, flexibility, and reduced complexity.
3. MongoDB was released on 27th August 2009. Version 1 was basic, while Version 2 introduced features like sharding, usable and spatial indices, and geospatial features.
4. Version 3 came with an aggregation framework, mainly as a supplement to the aging MapReduce framework.

★ **Exam Tip:** *MongoDB was released on 27th August 2009 — a common 2-mark question.*

2. What is MongoDB?

MongoDB is a non-relational (NoSQL) document database system. It is built on a horizontal scale-out architecture that uses a flexible schema for storing data.

Key Characteristics:

- 5. Instead of storing data in tables of rows/columns like SQL, each record in MongoDB is a document described in BSON — a binary representation of JSON data.
- 6. Document databases are highly flexible, allowing variations in the structure of documents and storing documents that are partially complete.
- 7. One document can have others embedded in it (nested documents).
- 8. Fields in a document play the role of columns in SQL databases and like columns, they can be indexed to increase search performance.
- 9. NoSQL databases store schema-less, unstructured data in multiple collections and nodes. Non-relational databases do not need fixed table schemas.
- 10. At the backend, MongoDB converts JSON data into binary format known as BSON.

2.1 SQL vs NoSQL Comparison

SQL (Relational)	NoSQL (MongoDB)
Uses Tables (rows & columns)	Uses Collections & Documents
Fixed schema required	Schema-less / flexible schema
Stores data in structured format	Stores data in BSON (Binary JSON)
ACID properties strictly enforced	Relaxed ACID for better performance
Vertical scaling	Horizontal scaling (sharding)
Examples: MySQL, Oracle, SQL Server	Example: MongoDB, Cassandra, Redis
Joins used for relationships	Embedded documents for relationships
Difficult to scale for Big Data	Excellent for large-scale data

3. Why MongoDB? (4 Major Reasons)

There are four major reasons why MongoDB is being deployed more often:

Reason 1: Flexibility

MongoDB uses documents that can contain sub-documents in complex hierarchies, making it expressive and flexible. MongoDB can map objects from any programming language, ensuring easy implementation and maintenance.

Reason 2: Flexible Query Model

The user can selectively index some parts of each document or query based on:

- Regular expressions
- Ranges
- Attribute values

They can have as many properties per object as needed by the application layer.

Reason 3: Native Aggregation

Native aggregation allows users to extract and transform data from the database. The data can either be loaded into a new format or exported to other data sources.

Reason 4: Schema-less Models

Applications get the power and responsibility to interpret different properties found in a collection's documents. This means different documents in the same collection can have different fields.

★ **Exam Tip:** *'Why MongoDB' is a classic 16-mark question. Cover all 4 reasons + salient features for full marks.*

4. Salient Features of MongoDB

Feature 1: General-Purpose Database

MongoDB can serve diverse sets of data and multiple purposes within a single application. It is not limited to a specific use case.

Feature 2: Flexible Schema Design

The document-oriented approach allows non-defined attributes to be modified on the fly. This is a key contrast between MongoDB and other RDBMS. There is no need to alter the entire database schema when a new field is added.

Feature 3: Load Balancing and Scalability

MongoDB is built to scale both vertically and horizontally using the technique of sharding:

- The architecture can achieve both write and read scalability.
- Data balancing occurs automatically and transparently to the user by the shard balancer.
- Sharding distributes data across multiple machines to handle large datasets.

Feature 4: Aggregation Framework

MongoDB offers an Extract, Transform, Load (ETL) framework which eliminates the need for complex data pipelines. It allows data processing and transformation within the database.

Feature 5: Native Replication

Data gets replicated across a replica set without a complicated setup. Replica sets provide redundancy and high availability of data. If the primary node fails, a secondary is automatically elected.

Feature 6: Security Features

Authentication and authorization are taken into account. MongoDB supports:

- Role-Based Access Control (RBAC)
- TLS/SSL encryption for data in transit
- Encryption at rest
- Auditing for compliance

Feature 7: JSON Support

JSON is widely used across frontend and API communication. It only makes sense for the database to use the same protocol. MongoDB natively stores data in a JSON-like format (BSON), making integration seamless.

Feature 8: MapReduce

MongoDB offers a great model (MapReduce) to build data pipelines. It allows for batch processing of data and aggregation operations across large datasets.

5. Working of MongoDB — Documents & Collections

5.1 How MongoDB Works

The entire database consists of Collections. These collections hold multiple documents. Since MongoDB is schemaless:

- The documents in one collection need not be similar.
- All data is stored as BSON documents with the help of key-value pairs.
- At the backend, MongoDB converts JSON data into binary format known as BSON.

5.2 What is a Document?

A document in MongoDB is a record in a collection. It is analogous to a row in a relational database. Documents are stored in BSON format (Binary JSON).

Example of a MongoDB Document:

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "username": "john_doe",
  "email": "john@example.com",
  "age": 25,
  "address": {
    "city": "Delhi",
    "pin": "110001"
  }
}
```

5.3 What is a Collection?

A collection is a group of MongoDB documents. It is the equivalent of a table in relational databases. Collections do not enforce a schema, so documents within a collection can have different fields.

5.4 Naming Restrictions for Collections

Before creating a collection, you should know the naming restrictions:

- Collection name must start with an underscore or a character.
- Collection name does not contain \$, empty string, null characters, and does not begin with 'system.' prefix.
- The maximum length of the collection name is 120 bytes.

5.5 Naming Restrictions for Fields

- Field names are strings.
- The `_id` field name is reserved for use as a primary key. The value of this field must be unique, immutable, and can be of any type other than arrays.
- Field names cannot contain null characters.
- Top-level field names should not start with dollar sign (\$).

5.6 Naming Restrictions for Database

- In MongoDB, the names of the database are case-insensitive, but you must always remember that the database names cannot differ only by the case of characters.

- For Windows users, MongoDB database names cannot contain any of the following characters: / \ . " \$ * < > : | ?

5.7 Document Size

The max size of the BSON document is 16MB. It ensures that a single document does not use too much RAM or bandwidth.

5.8 Important Fields — _id Field

In MongoDB, every document stored in the collection must contain a unique _id field. It is just like a primary key in a relational database.

- When you create a collection, MongoDB automatically creates a unique index on the _id field.
- The _id field is the first field in every document.
- The value of the _id field can be of any BSON type except arrays.
- The default value of the _id field is ObjectId.

 **Note:** *A single document may contain duplicate fields (same field name more than once). MongoDB always saves the order of the fields in the document except for the _id field.*

6. Data Model Design

With MongoDB, you may embed related data in a single structure or document. The two approaches to data modeling are:

- Embedded Data Models (Denormalized)
- Normalized Data Models (Referenced)

6.1 Embedded Data Models (Denormalized Models)

These schemas are generally known as denormalized models and take advantage of MongoDB's rich document capabilities.

Definition:

Embedded data models allow applications to store related pieces of information in the same database record. Related data is embedded within a single document structure.

When to use Embedded Data Models:

11. We have 'contains' relationships between entities.
12. We need to model one-to-one relationships with embedded documents.
13. We have model one-to-many relationships with embedded documents.

Advantages:

- Embeddings provide better performance for read operations as well as the ability to request and retrieve related data in a single database operation.
- Atomic write operations — updating embedded data happens in one operation.
- Better performance for read-heavy applications.

Disadvantage — Document Size Limit:

Documents in MongoDB must be smaller than the max BSON document size (16MB). If embedded documents grow too large, you may need to use GridFS or normalize the data.

Example of Embedded Data Model:

A user document with embedded contact and access information:

```
{
  "_id": ObjectId("507f1f77"),
  "username": "123xyz",
  "contact": {
    "phone": "123-456-7890",
    "email": "xyz@example.com"
  },
  "access": {
    "level": 5,
    "group": "dev"
  }
}
```

 **Note:** The 'contact' and 'access' objects are embedded sub-documents inside the main user document.

How to Access Embedded Data:

To access data within embedded documents, use dot notation to reach into embedded documents:

```
db.users.find({ 'contact.phone': '123-456-7890' })
db.users.find({ 'access.level': 5 })
```

6.2 Normalized Data Models (Referenced Models)

Normalized data models describe relationships using references between documents (similar to foreign keys in relational databases).

When to use Normalized Data Models:

- 14. When embedding would result in duplication of data but could not provide sufficient read performance advantages to outweigh the implications of the duplication.
- 15. To represent more complex many-to-many relationships.
- 16. To model large hierarchical data sets.

Example of Normalized Data Model:

Instead of embedding contact info, store it in a separate collection and reference it:

```
// User Document
{
  "_id": ObjectId("507f1f77"),
  "username": "123xyz"
}

// Contact Document (separate collection)
{
  "_id": ObjectId("507f1f78"),
  "user_id": ObjectId("507f1f77"),
  "phone": "123-456-7890",
  "email": "xyz@example.com"
}

// Access Document (separate collection)
{
  "_id": ObjectId("507f1f79"),
  "user_id": ObjectId("507f1f77"),
  "level": 5,
  "group": "dev"
}
```

6.3 Embedded vs Normalized — Comparison

Embedded Data Model	Normalized Data Model
Data stored in one document	Data split across multiple documents
Faster read operations (single query)	Multiple queries needed to fetch related data
Suitable for one-to-one and one-to-many	Suitable for many-to-many relationships
Risk of 16MB document size limit	No document size concern
Data duplication possible	No data duplication
Denormalized structure	Normalized structure
Easy atomic updates	Requires multi-document transactions
Best for: user profiles, blog posts	Best for: social networks, e-commerce

7. MongoDB Use Cases

Use Case 1: Product Data Management

MongoDB is perfect for product data management. It enables product data and related information to be managed and processed in a single, central system. This allows for:

- Detailed cost analysis
- Increased productivity
- Improved collaboration

Use Case 2: Operational Intelligence

Another real-world MongoDB use case is Operational Intelligence as it aids in real-time decision making. It allows companies to seamlessly gather various data feeds representing their ongoing business operations and info of related external factors.

Use Case 3: Product Catalog

Product catalogs have been in existence for years in the ever-evolving digital space. In the rapid technology sector, product catalogs sometimes feel like a new digital experience because of the richness and volume of data feed product catalogs. Its dynamic schema capability ensures that product elements only contain attributes relevant to the product.

Use Case 4: Scaling and Application Mobility

For most mobile application developers, it is expected that companies involved will have to deal with different data structures from several sources and potentially highly dynamic growth. MongoDB provides high flexibility and scalability that serves as an excellent database solution for such challenges.

Use Case 5: Customer Analytics

Data aggregation allows many companies to create exceptional customer experiences because they can collect massive amounts of data of their potential and existing customers and add it with publicly available data.

Use Case 6: Mainframe Offloading

Despite high operational cost and new databases dominating the market, mainframe remains essential in infrastructure because moving data off the mainframe is a challenging task. MongoDB is efficient in offloading data from mainframe systems at reduced operational cost.

Use Case 7: Real-Time Data Integration

Companies usually have amounts of data distributed across their organization. These data are valuable when aggregated in a single view. MongoDB provides flexibility and query capabilities that make aggregating and organizing data more efficient.

8. Real-World Examples of MongoDB

1. Aadhaar (India)

India's Unique Identification project boasts the largest biometric database in the world. The Aadhaar project uses MongoDB to store the massive demographic and biometric data of over 1.2 billion Indians.

2. eBay

The popular online auction site uses MongoDB for projects such as cloud management, metadata storage, and search suggestions.

3. Shutterfly

This is a well-known internet-based photo sharing company that has over 6 billion images and a transaction rate up to 10,000 operations per second. MongoDB handles this massive scale efficiently.

4. Electronic Arts (EA)

EA uses MongoDB for FIFA Online 3, their popular online multiplayer game.

5. Forbes

When a story becomes viral, people resort to every available website to get details. Therefore it is essential for publishers to be alert and give sharable info as soon as possible to keep their readers informed. Forbes uses MongoDB for this purpose.

6. MetLife

MetLife is a market leader in employee insurance, benefit plans, and pensions. They serve over 90 million clients across Middle East, Asia, etc. MetLife's sophisticated customer service solution 'The Wall' is built on MongoDB.

7. Otto

Many retail businesses need real-time analytics. This requires that their apps remain relevant and always available online with no interruption. This is because slow response time leads to a lot of revenue loss in the retail industry. Otto uses MongoDB for this real-time capability.

8. Weather Channel

The Weather Channel had problems servicing a massive client because it used an RDBMS system. However, MongoDB enabled them to develop their smartphone application that serves over 40 million active users and provides real-time meteorological data to its clients. They also have MongoDB's scalable solution and MapReduce functionalities.

9. Limitations of MongoDB

- MongoDB uses high memory for data storage.
- The BSON document size cannot exceed 16MB.
- Naming restrictions for databases on Windows — cannot contain special characters like /, \, ., \$, *, <, >, :, |, ?.

- MongoDB does not support complex multi-document ACID transactions (though MongoDB 4.0+ added multi-document transactions).
- Joins are not natively supported the way SQL supports them — you need \$lookup in aggregation.
- No support for stored procedures like SQL.

EXAMINATION PREPARATION

PART A: 2-Mark Questions with Answers (10 Questions)

Q1. What is MongoDB? When was it released?

MongoDB is a non-relational (NoSQL) document database system built on a horizontal scale-out architecture that uses a flexible schema for storing data. Each record is stored as a BSON document (binary representation of JSON). MongoDB was released on 27th August 2009. Version 1 was basic; Version 2 introduced sharding, spatial indices, and geospatial features.

Q2. What is BSON? How is it different from JSON?

BSON (Binary JSON) is a binary representation of JSON-like documents. At the backend, MongoDB converts JSON data into binary format known as BSON. Differences:

- JSON is text-based; BSON is binary-encoded.
 - BSON supports additional data types not in JSON, such as Date, Binary data, and ObjectId.
 - BSON is faster to parse and more efficient for storage.
 - BSON allows MongoDB to efficiently encode and decode documents.
-

Q3. What is the maximum size of a BSON document in MongoDB?

The maximum size of a BSON document in MongoDB is 16MB. This limit ensures that a single document does not use too much RAM or bandwidth. If documents need to store data that exceeds this limit, MongoDB uses GridFS API to divide and store the content.

Q4. What is the `_id` field in MongoDB?

In MongoDB, every document stored in the collection must contain a unique `_id` field. It is just like a primary key in a relational database. Key properties:

- It is the first field in every document.
 - Its value must be unique, immutable, and can be of any BSON type except arrays.
 - The default value is ObjectId (a 12-byte identifier).
 - When a collection is created, MongoDB automatically creates a unique index on the `_id` field.
-

Q5. What is a Collection in MongoDB?

A collection in MongoDB is a group of documents, analogous to a table in relational databases. Key features:

- Collections do not enforce a schema — documents in the same collection can have different fields.
 - Collection name must start with an underscore or a character.
 - Collection name must not contain \$, empty string, null characters, or begin with 'system.' prefix.
 - Maximum length of collection name is 120 bytes.
-

Q6. What are the two types of Data Models in MongoDB?

MongoDB supports two types of data models:

- Embedded Data Model (Denormalized): Related data is stored within a single document as sub-documents. Best for one-to-one and one-to-many relationships. Provides better read performance.
 - Normalized Data Model (Referenced): Data is stored in separate documents with references between them (like foreign keys). Best for many-to-many relationships and large hierarchical datasets.
-

Q7. What is the difference between SQL and NoSQL databases?

SQL databases store data in structured tables with fixed schemas using rows and columns (e.g., MySQL, Oracle). NoSQL databases like MongoDB store schema-less, unstructured data in multiple collections and nodes. NoSQL does not need fixed table schemas, relaxes ACID properties for better performance and scalability, and is better suited for handling large-scale, unstructured data.

Q8. What is Sharding in MongoDB?

Sharding is a method for distributing data across multiple machines (horizontal scaling). MongoDB uses sharding to support deployments with very large datasets and high throughput operations. Key points:

- Data balancing occurs automatically and transparently to the user by the shard balancer.
 - It achieves both write and read scalability.
 - Sharding splits data across multiple shards (servers), each acting as an independent database.
-

Q9. What is a Replica Set in MongoDB?

A replica set in MongoDB is a group of MongoDB instances that maintain the same dataset, providing redundancy and high availability.

- Data gets replicated across a replica set without a complicated setup.
 - A replica set consists of a primary node and secondary nodes.
 - If the primary node fails, a secondary is automatically elected as the new primary.
 - This ensures data is always available even in case of hardware failure.
-

Q10. State three naming restrictions for MongoDB databases.

Naming restrictions for MongoDB databases:

17. Database names are case-insensitive, but cannot differ only by the case of characters (e.g., 'MyDB' and 'mydb' cannot coexist).
18. For Windows users, MongoDB database names cannot contain: / \ . " \$ * < > : | ?
19. The database name 'admin', 'local', and 'config' are reserved system databases.

Q1. Explain the need for MongoDB, its salient features, and why it is preferred over traditional RDBMS. (16 Marks)

Introduction (2 marks):

In a data-driven world, we need organized and easily accessible information. A database is any structured information specially organized and stored in a computer for fast retrieval and search. Traditional SQL databases store data in tables with fixed schemas, which became limiting as web applications grew in complexity and scale. MongoDB was developed as a NoSQL solution to address these limitations, released on 27th August 2009.

Why MongoDB? — 4 Major Reasons (4 marks):

1. Flexibility:

MongoDB uses documents that can contain sub-documents in complex hierarchies, making it expressive and flexible. It can map objects from any programming language, ensuring easy implementation and maintenance.

2. Flexible Query Model:

The user can selectively index some parts of each document or query based on regular expressions, ranges, or attribute values, and have as many properties per object as needed by the application layer.

3. Native Aggregation:

Native aggregation allows users to extract and transform data from the database. The data can be loaded into a new format or exported to other data sources, replacing complex ETL pipelines.

4. Schema-less Models:

Applications get the power and responsibility to interpret different properties found in collection documents. Different documents in the same collection can have different fields.

Salient Features of MongoDB (8 marks):

1. General-Purpose Database:

MongoDB can serve diverse sets of data and multiple purposes within a single application.

2. Flexible Schema Design:

The document-oriented approach allows non-defined attributes to be modified on the fly. This is a key contrast between MongoDB and other RDBMS — no schema migration needed.

3. Load Balancing and Scalability:

Built to scale both vertically and horizontally using the technique of sharding. Data balancing occurs automatically and transparently by the shard balancer, achieving both write and read scalability.

4. Aggregation Framework:

MongoDB offers an ETL (Extract, Transform, Load) framework which eliminates the need for complex data pipelines, allowing data processing and transformation within the database itself.

5. Native Replication:

Data gets replicated across a replica set without a complicated setup, providing redundancy and high availability. Secondary nodes are automatically elected if the primary fails.

6. Security Features:

MongoDB supports Role-Based Access Control (RBAC), TLS/SSL encryption for data in transit, encryption at rest, and auditing for compliance.

7. JSON/BSON Support:

JSON is widely used in frontend and API communication. MongoDB natively stores data in BSON (Binary JSON), making integration with modern web applications seamless.

8. MapReduce:

MongoDB offers MapReduce to build data pipelines for batch processing and aggregation operations across large datasets.

Conclusion (2 marks):

MongoDB is preferred over traditional RDBMS because of its flexible schema, superior scalability through horizontal sharding, better performance for unstructured and semi-structured data, native JSON support, and rich aggregation capabilities. These features make it ideal for modern web applications, big data, and real-time analytics.

Q2. Explain Documents and Collections in MongoDB in detail. What are the naming conventions and restrictions? Explain the `_id` field with examples. (16 Marks)

Introduction (1 mark):

MongoDB stores data as BSON documents organized within collections. Unlike relational databases, MongoDB is schemaless, meaning documents in the same collection can have different structures.

What is a Document? (3 marks)

A document in MongoDB is the basic unit of data, analogous to a row in a relational database. It is stored in BSON (Binary JSON) format — a binary representation of key-value pairs.

Key properties of documents:

- A single document may contain duplicate fields.
- MongoDB always saves the order of fields in the document except for the `_id` field.
- Documents are highly flexible — one document can have others embedded in it.
- The maximum BSON document size is 16MB.

Example Document:

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "username": "john_doe",
  "age": 28,
  "address": { "city": "Delhi", "pin": "110001" },
  "hobbies": ["cricket", "coding"]
}
```

What is a Collection? (2 marks)

A collection is a group of MongoDB documents, equivalent to a table in relational databases. Collections do not enforce a schema — documents within a collection can have different fields and data types.

Naming Restrictions for Collections (2 marks):

20. Collection name must start with an underscore or a letter.
21. Collection name must not contain: \$ symbol, empty string, null characters.
22. Collection name must not begin with the 'system.' prefix (reserved).
23. Maximum length of collection name is 120 bytes.

Naming Restrictions for Fields (2 marks):

24. Field names are strings.
25. The `_id` field name is reserved as the primary key — value must be unique, immutable, and any BSON type except arrays.
26. Field names cannot contain null characters.
27. Top-level field names should not start with a dollar sign (\$).

Naming Restrictions for Databases (2 marks):

28. Database names are case-insensitive but cannot differ only by case.
29. Windows restrictions — names cannot contain: / \ . \$ * < > : | ?
30. System databases: 'admin', 'local', 'config' are reserved.

The `_id` Field in Detail (4 marks):

In MongoDB, every document stored in a collection must contain a unique `_id` field. It functions as the primary key.

- When you create a collection, MongoDB automatically creates a unique index on the `_id` field.
- The `_id` field is always the first field in every document.
- Its value can be of any BSON type except arrays.
- The default value of the `_id` field is `ObjectId` — a 12-byte identifier composed of: timestamp (4 bytes), machine identifier (3 bytes), process id (2 bytes), and counter (3 bytes).
- You can supply your own `_id` value as long as it is unique within the collection.

Example showing `ObjectId` structure:

```
ObjectId("507f1f77bcf86cd799439011")  
// Breakdown:  
// 507f1f77 = timestamp (Unix epoch)  
// bcf86c   = machine identifier  
// d799     = process ID  
// 439011   = counter (auto-incrementing)
```

Q3. Explain Embedded Data Models and Normalized Data Models in MongoDB with examples. When should each be used? (16 Marks)

Introduction — Data Model Design (2 marks):

With MongoDB, you may embed related data in a single structure (embedded model) or reference related data across separate documents (normalized model). Choosing the right data model significantly impacts the performance and scalability of your application.

1. Embedded Data Models — Denormalized (7 marks)

Definition:

Embedded data models allow applications to store related pieces of information in the same database record. These schemas are known as denormalized models and take advantage of MongoDB's rich document capabilities.

When to Use Embedded Models:

31. When we have 'contains' relationships between entities (e.g., a person and their address).
32. To model one-to-one relationships with embedded documents.
33. To model one-to-many relationships where the 'many' objects always appear with the 'one' object.

Advantages:

- Better performance for read operations — all data retrieved in a single database operation.
- Atomic writes — updating an embedded document happens in a single write operation.
- Reduced number of queries.

Disadvantage:

- Document size limit: Documents in MongoDB must be smaller than the max BSON doc size (16MB). Growing embedded data can violate this limit.

Example of Embedded Model:

```
// One document containing everything
{
  "_id": ObjectId("507f1f77"),
  "username": "123xyz",
  "contact": {
    "phone": "123-456-7890",
    "email": "xyz@example.com"
  },
  "access": {
    "level": 5,
    "group": "dev"
  }
}
```

Accessing embedded documents using dot notation:

```
db.users.find({ 'contact.phone': '123-456-7890' })
db.users.find({ 'access.level': { $gte: 3 } })
```

2. Normalized Data Models — Referenced (5 marks)

Definition:

Normalized data models describe relationships using references between documents — similar to foreign keys in relational databases. Instead of embedding, data is stored in separate collections and linked via ObjectId references.

When to Use Normalized Models:

34. When embedding would result in duplication of data but could not provide sufficient read performance advantages to outweigh the duplication implications.

- 35. To represent more complex many-to-many relationships.
- 36. To model large hierarchical data sets.

Example of Normalized Model:

```
// User document (user collection)
{ "_id": ObjectId("507f1f77"), "username": "123xyz" }

// Contact document (contacts collection)
{
  "_id": ObjectId("507f1f78"),
  "user_id": ObjectId("507f1f77"),
  "phone": "123-456-7890",
  "email": "xyz@example.com"
}

// Access document (access collection)
{
  "_id": ObjectId("507f1f79"),
  "user_id": ObjectId("507f1f77"),
  "level": 5,
  "group": "dev"
}
```

Comparison Table (2 marks):

Embedded (Denormalized)	Normalized (Referenced)
Related data in one document	Data split across collections
Single query retrieval	Multiple queries / \$lookup needed
One-to-one, one-to-many	Many-to-many, hierarchical
Risk of 16MB size limit	No size concern
Data may be duplicated	No data duplication
Faster reads	Slower reads, less storage

Q4. Explain any seven real-world use cases of MongoDB with examples. (16 Marks)

Introduction (1 mark):

MongoDB's flexible schema, horizontal scalability, and rich query capabilities make it suitable for a wide variety of real-world applications. The following are prominent use cases and examples:

Use Case 1: Product Data Management (2 marks)

MongoDB is perfect for product data management. It enables product data and related information to be managed and processed in a single, central system. This allows for detailed cost analysis, increased productivity, and improved collaboration. Its flexible schema means different products can have different attributes without schema changes.

Use Case 2: Operational Intelligence (2 marks)

Operational Intelligence aids in real-time decision making. MongoDB allows companies to seamlessly gather various data feeds representing their ongoing business operations and information of related external factors. Example: tracking live inventory, sales, and supply chain data in real time.

Use Case 3: Product Catalog (2 marks)

Product catalogs in e-commerce have varied and evolving data. Its dynamic schema capability ensures that product elements only contain attributes relevant to the product. For example, a shirt has size and color, while a laptop has RAM and processor — different fields, same collection.

Use Case 4: Scaling and Application Mobility (2 marks)

For mobile application developers, companies face different data structures from several sources and highly dynamic growth. MongoDB provides high flexibility and scalability that serves as an excellent database solution. Example: a food delivery app scaling during peak hours using MongoDB's sharding.

Use Case 5: Customer Analytics (2 marks)

Data aggregation allows many companies to create exceptional customer experiences because they can collect massive amounts of data of their potential and existing customers and combine it with publicly available data. MongoDB's aggregation pipeline processes this data efficiently.

Use Case 6: Mainframe Offloading (2 marks)

Despite high operational cost, mainframes remain essential in infrastructure. Moving data off the mainframe is challenging. MongoDB is efficient in offloading data from mainframe systems at reduced operational cost, allowing legacy systems to interact with modern architectures.

Use Case 7: Real-Time Data Integration (3 marks)

Companies usually have data distributed across their organization. These data are valuable when aggregated in a single view. MongoDB provides flexibility and query capabilities that make aggregating and organizing data more efficient. Example: Weather Channel — struggled with RDBMS but switched to MongoDB to serve over 40 million active users with real-time meteorological data. Also leveraged MongoDB's scalable solution and MapReduce functionalities.

Other Notable Real-World Examples:

- Aadhaar: Stores demographic and biometric data of 1.2 billion Indians — the world's largest biometric database.
- eBay: Uses MongoDB for cloud management, metadata storage, and search suggestions.
- Shutterfly: Manages 6 billion images with 10,000 operations per second.
- MetLife: Built 'The Wall' customer service solution for 90 million clients on MongoDB.

Q5. Explain the evolution of MongoDB and its working. Describe the structure of MongoDB including collections, documents, fields, and BSON with examples. (16 Marks)

1. Evolution of MongoDB (4 marks)

Pre-MongoDB Era:

SQL existed even before the World Wide Web. As websites grew in functionality, developers wanted to generate web pages using content that could change over time without re-deploying the code. This demand created the need for a more flexible database.

Rise of NoSQL:

NoSQL databases relaxed ACID (Atomicity, Consistency, Isolation, Durability) properties and guaranteed better performance, scalability, flexibility, and reduced complexity compared to traditional SQL systems.

MongoDB Timeline:

37. 2009 — MongoDB released on 27th August 2009 by 10gen (now MongoDB Inc.). Version 1 was basic.
38. Version 2 — Introduced features like sharding, usable and spatial indices, and geospatial features.
39. Version 3 — Came with an aggregation framework, mainly as a supplement to the aging MapReduce framework.
40. Version 4.0+ — Added multi-document ACID transactions, making MongoDB suitable for even more enterprise use cases.

2. What is MongoDB? (2 marks)

MongoDB is a non-relational (NoSQL) document database system. It is built on a horizontal scale-out architecture that uses a flexible schema for storing data. Instead of storing data in tables of rows/columns like SQL, each record in MongoDB is a document described in BSON (Binary JSON).

3. Working of MongoDB — Overall Structure (4 marks)

The entire database consists of Collections. These collections hold multiple documents. Since MongoDB is schemaless:

- The documents in one collection need not be similar.
- All data is stored as BSON documents with the help of key-value pairs.
- At the backend, MongoDB converts JSON data into binary format known as BSON.

Hierarchy of MongoDB:

```
MongoDB Server
├── Database (e.g., mydb)
│   ├── Collection (e.g., users)
│   │   ├── Document (e.g., { _id: ..., name: 'John', age: 25 })
│   │   │   └── Fields (key-value pairs)
```

4. Documents — Structure and Properties (2 marks)

A document is the basic unit of data in MongoDB, equivalent to a row in a relational database. It is stored in BSON format.

```
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "name": "Priya Sharma",
  "age": 22,
  "courses": ["MongoDB", "Python", "Machine Learning"],
  "address": {
    "city": "Delhi",
    "state": "Delhi",
```

```
"pin": "110001"  
},  
"enrolled": true  
}
```

5. Collections (1 mark)

A collection is a group of documents. It is equivalent to a table in RDBMS. Collections are schemaless — documents within can have different fields.

6. BSON — Binary JSON (2 marks)

BSON stands for Binary JSON. It is the binary-encoded serialization of JSON-like documents that MongoDB uses internally.

- BSON extends JSON with additional types: Date, Binary data, ObjectId, Regular expression, JavaScript, etc.
- At the backend, MongoDB converts JSON data into binary format known as BSON for efficient storage.
- BSON is faster to encode/decode and allows MongoDB to work efficiently.
- The maximum BSON document size is 16MB to ensure single documents don't monopolize RAM or bandwidth.

7. The `_id` Field (1 mark)

Every document must have a unique `_id` field (primary key). Default value is ObjectId — a 12-byte unique identifier automatically generated by MongoDB when you insert a document without specifying `_id`.

QUICK REVISION — KEY FACTS

Topic	Key Fact to Remember
MongoDB Release	27th August 2009 by 10gen (now MongoDB Inc.)
Document Format	BSON (Binary JSON)
Max Document Size	16 MB
Primary Key	<code>_id</code> field (default: ObjectId, 12 bytes)
Max Collection Name Length	120 bytes
Collection starts with	Underscore (<code>_</code>) or a character
NoSQL Properties	Relaxed ACID — better performance, scalability, flexibility
Horizontal Scaling	Achieved through Sharding
High Availability	Achieved through Replica Sets
Data Models	Embedded (Denormalized) and Normalized (Referenced)
When to Embed	One-to-one or one-to-many relationships
When to Normalize	Many-to-many or large hierarchical datasets
Aggregation Framework	ETL — Extract, Transform, Load within DB
Largest User	Aadhaar — 1.2 billion Indians' biometric data
4 Reasons for MongoDB	Flexibility, Flexible Query Model, Native Aggregation, Schema-less Models
8 Salient Features	General-purpose, Flexible Schema, Load Balancing, Aggregation, Replication, Security, JSON, MapReduce

All the Best for Your Examinations!

Unit 2 — Introduction to MongoDB | End Semester Notes